

#

Requirement elicitation - techniques

1. Facilitated appⁿ Specification technique? (FAST)

The Objective is to bridge the expectation gap - the difference b/w what the developers think they are supposed to build & what customers think they are going to get. A team oriented approach is developed for requirements gathering.

each attendee is asked to make a list of objects that are -

- Part of environment that surrounds the System.
- produced by the system
- used by the system.

each participant prepares list, different lists are then combined, redundant entries are eliminated, team is divided into smaller sub-teams to develop mini specifications & finally a draft of specifications is written down using all the inputs from the meeting.

2. > Quality Function Deployment (QFD) :

In this technique Customer Satisfaction is prime concern, hence 3 types of requirements are identified -

- Normal requirements

In this, the Objectives & Goals of proposed S/W are discussed with Customer.

- expected requirements

These requirements are so obvious that the Customer Need not explicitly state them.

e.g protection from unauthorized access.

- exciting requirements

it includes features that are beyond Customer's expectations & prove to be very Satisfying when present.

e.g when unauthorized access is detected, it should backup & Shutdown all processes.

The Major steps involved in this procedure are —

1. identify all the users, developers, customers.
2. List out all requirements from Customer
3. A Value of Importance is assigned to each requirement.
4. In the end the final list of requirements is prepared.

3. > Use Case Approach

This technique combines text & pictures to provide a better understanding of the requirements.

The use cases describe the 'what' of a system & not 'how'. Hence, they only give a functional view of system.

The Components are:

- ① Actor: it is the external agent that lies outside the system but interacts with it in some way. An actor may be a person, machine etc.

2. Use Cases: They describe the Sequence of interactions b/w actor & System. They captures actors and interaction with the System.

3. Use Case diagram: it Graphically represents what happens when an actor interacts with a System. it Capture only functional aspect of the System.

Requirement Analysis

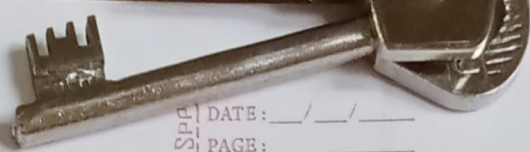
Data dictionaries

it is the Major Component in the Structured Analysis Model of System. it lists all the data items appearing in DFD (data flow diagram). A data dictionary is a file or a Set of files that includes a Database's Metadata, like data ownership, relationship of data to another object and some other data.

e.g: a data dictionary entry:

$$\text{Gross pay} = \text{Regular pay} + \text{Overtime pay}$$

Case tool is used to Maintain data dictionary as it captures



The data items appearing in a DFD automatically to generate the data dictionary.

⇒ Components

1. Name of item: it can be your choice
2. Description: Text
3. related data items: with other data items.
4. Range of Values: it will represent all possible answers.

⇒ Data dictionary Notations

<u>notations</u>	<u>Meaning</u>
• $X = a + b$	X Consists data elements a & b
• $X = [a b]$	X Consists of either elements a or b .
• $X = a X$	X Consists of optional data elements a
• $X = y [a]$	X Consists of y or more events of data element a .

The data items appearing in a DFD automatically to generate the data dictionary.

⇒ Components

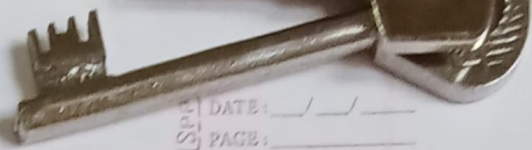
1. Name of item : it can be your choice
2. Description : Text
3. related data items : with other data items.
4. Range of Values : it will represent all possible answers.

⇒ Data dictionary Notations

notations

Meaning

- $X = a + b$ X Consists data elements a & b
- $X = [a | b]$ X Consists of either elements a or b .
- $X = a X$ X Consists of optional data elements a
- $X = y[a]$ X Consists of y or more events of data element a .



-The data items appearing in a DFD automatically to generate the data dictionary.

⇒ Components

1. name of item : it can be your choice
2. Description : Text
3. related data items : with other data items.
4. range of values : it will represent all possible answers.

⇒ Data dictionary Notations

notations

Meaning

• $X = a + b$

X Consists data elements a & b

• $X = [a | b]$

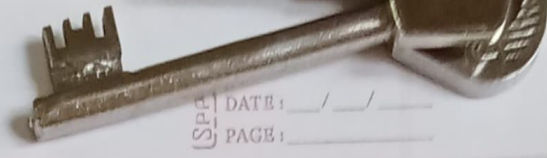
X Consists of either elements a or b.

• $X = aX$

X Consists of optional data elements a

• $X = y[a]$

X Consists of y or more events of data element a.



-The data items appearing in a DFD automatically to generate the data dictionary.

⇒ Components

1. Name of item : it can be your choice
2. Description : Text
3. related data items : with other data items.
4. range of values : it will represent all possible answers.

⇒ Data dictionary Notations

<u>notations</u>	<u>Meaning</u>
• $X = a + b$	X Consists data elements a & b
• $X = [a b]$	X Consists of either elements a or b.
• $X = a X$	X Consists of optional data elements a
• $X = y [a]$	X Consists of y or more events of data element a.

- $X = [a]z$ X Consists of z or less events of data element a
- $X = y[a]z$ X Consists of Some events of data elements b/w y & z .

⇒ Features of Data dictionary

- It helps in designing test cases & designing the sw.
- It is Very Important for Creating an ordered list from a Subset of the items list.
- It is Very Important for Creating an Ordered list from a Complete items list.
- The data dictionary is also important to find the specific data item object from the list.

⇒ Uses

1. It is used for Creating the Ordered list of data items.

2. Used for finding data items from a description in s/w engineering.

⇒ Importance

1. It provides developers with standard terminology for all data.
2. It provides developers to use different terms to refer to the same data.
3. It provides definitions for different data.
4. Query handling is facilitated if a data dictionary is used in RDBMS.

⇒ Advantage

- ① A data dictionary can help improve data quality by providing a single source of truth for data definitions, allowing users to easily verify the accuracy & completeness of data.
- ② It helps to improve efficiency by reducing the time and effort required to define, document & manage data elements & attributes.

⇒ Disadv.

- ① Implementation & Maintenance Cost is high.
- ② Increase Complexity.
- ③ Do Not Gives Data Security.
- ④ More Difficult to Use.

Requirements Analysis

what is an Entity-Relationship diagram (ERD).

It is a Graphical representation of data model using entities, their attributes and relationships b/w those entities. It has a form of diagram.

Adv.

- ① easier to See the big picture
- ② easier to understand table relations

Disadv.

- ① Doesn't work with large data Models.

Data flow diagrams

DfD are used for Modeling the requirements. They have been used for many years prior to the advent of computers. DfDs show the flow of data through a System.

The data may be a Company, an Organisation, a Set of procedure, a Computer hardware System, a S/W System or any combination of the preceding.

The DfD is also known as a data flow graph or a bubble chart.

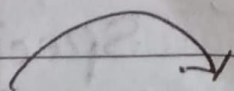
Note:

1. All the Names should be unique.
2. DfD is not a flow chart.
3. Defect error condⁿ & error handling until the end of the analysis.

Symbol

Name

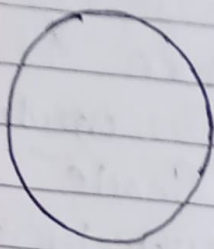
funcⁿ



Data
flow

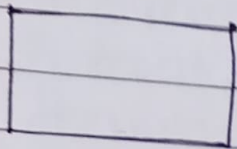
Used to connect processes to each other,

to Sources or Sinks, the arrow head indicates direction of data flow.



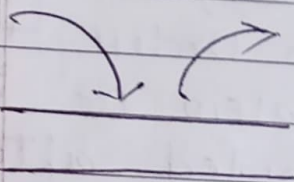
Process

performs some transformation of I/p data to yield of data.



Source

A source of system inputs.



Data Store

A repository of data; the arrow heads indicates net i/p & net o/p to store.

slw prototyping

Prototyping is the technique of constructing a partial implementation of a system so that customers, users or developers can learn more about a problem or a solⁿ to that problem.

it is partial Implementation bcz if it were full implementation, it would be the System, not a prototype of it.

it allows user to explore & Criticize proposed Systems before undergoing the cost of a full Scale development. The field of prototyping s/w Systems has emerged around 2 prototyping technologies -

1. throw away
2. evolutionary.

In throw-away approach, the prototype s/w is constructed in order to learn about the problem or its solⁿ and is usually discarded after the desired knowledge is gained.

In evolutionary approach, the prototype is constructed in order to learn about the problem or its solⁿ in successive steps.

benefits of Developing a prototype early in the s/w process are -

1. Misunderstanding b/w s/w developers and Customers May be identified as the System funcⁿ are demonstrated.

2. Missing user requirements may be detected.

3. it is a base for writing the specification of the system.